

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

Кафедра вычислительной техники



Курсовая работа
по дисциплине «Технология разработки программного
обеспечения»

Тема: «Система автоматизации диспетчерской службы такси.
Серверная часть»

Группа: АПИМ-16
Студенты: Дементьев А..А.
Преподаватель: Романов Е.Л.

Новосибирск
2017

Оглавление

1	Задания к курсовой работе.....	4
2	Анализ предметной области	5
2.1	Глоссарий	5
2.2	Словесное описание бизнес-процессов предметной области	5
2.3	Бизнес-требования.....	5
2.4	Перечень пользователей проекта	6
2.5	Диаграмма классов предметной области	6
3	Модели прецедентов и спецификации требований.....	6
3.1	Актеры (роли) и прецеденты.....	6
3.2	Особенности и требования к реализации.....	7
3.3	Пример сценария	8
4	Спецификации требований	9
4.1	Бизнес-правила.....	9
4.2	Атрибуты качества	10
4.3	Функциональные требования	10
4.4	Отчеты	10
4.5	Графический интерфейс	10
5	Архитектура системы	11
5.1	Многослойность архитектуры ПО.....	11
5.2	Представление развёртывания	12
5.3	Процедурное представление	12
5.4	Логическое представление	12
5.5	Реализационное представление.....	12
5.6	Параллелизм. Синхронизация.....	13
5.7	Взаимодействие и представление (MVC)	13
5.8	Диаграмма компонентов	14
6	Прототип программной реализации	15
6.1	База данных	15
6.2	Структура приложения	19

Заключение.	25
Список литературы	26
Листинг программы	27

1 Задания к курсовой работе

Система автоматизации диспетчерской службы такси. Серверная часть.

Использую современные инструменты и паттерны программирования разработать серверную часть для приложения Диспетчера службы такси.

2 Анализ предметной области

2.1 Глоссарий

- Директор
- Диспетчер – человек, принимающий заявки в службе такси.
 - Имя
 - Смена
- Водитель – принимает заказ на выполнение
- Клиент
- Автомобиль
 - Тип кузова
 - Марка
- Заявка – принятый заказ от клиента.
- Очередь – некоторое число заявок, принятых на выполнение.
- Приложение водителя – приложение для быстрого доступа к очереди заявок.
- Адресная база – хранятся адреса прибытия и отбытия.
- Оплата заказа – в нее входят стоимость самой поездки, стоимость ожидания такси и дополнительные услуги.

2.2 Словесное описание бизнес-процессов предметной области

Некоторая компания N предлагает свои услуги такси. Чтобы сделать заказ, необходимо связаться с диспетчером.

Диспетчер принимает заказ, при этом уточняя место и время подачи машины. Далее его задача состоит в том, чтобы поставить этот заказ в очередь на исполнение в приложении для водителей.

Водитель же, выбрав заказ через приложение, получает данные о клиенте (адрес отправления). Выполнив заказ, водитель получает оплату. Если же клиенту потребовались дополнительные услуги, водитель может предоставить их за отдельную плату.

2.3 Бизнес-требования

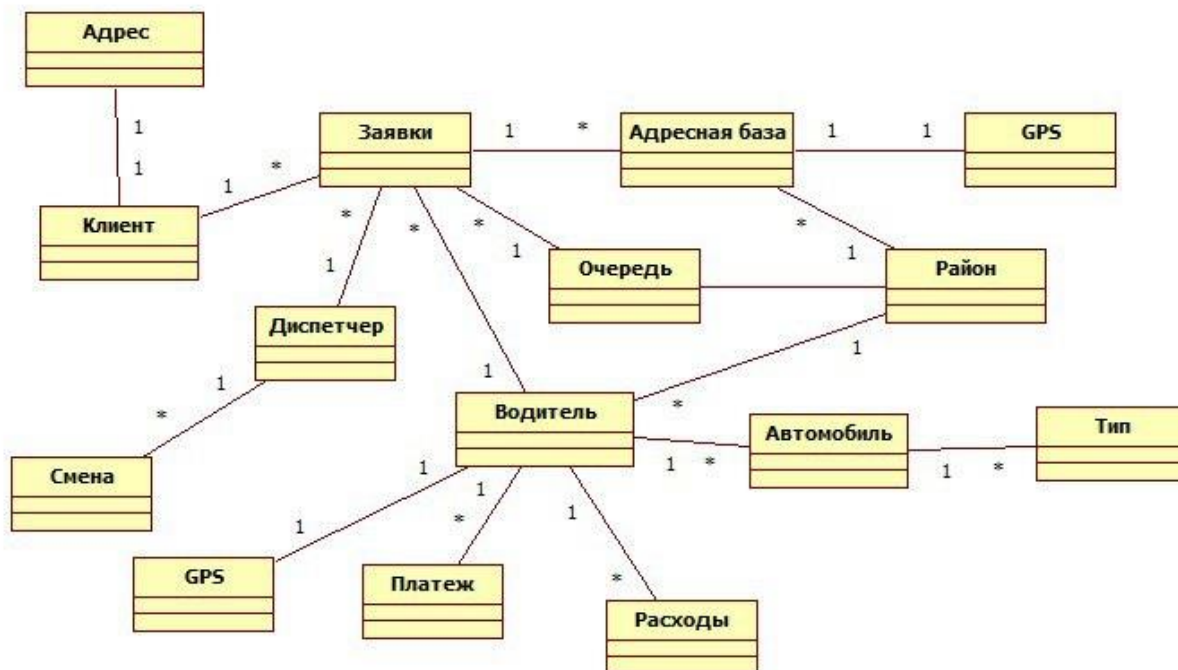
- ✓ Обеспечить всех работников(водителей) службы такси gps-навигатором, для удобства водителей и клиентов;
- ✓ Установить в автомобилях устройства безналичного расчета;
- ✓ Продажа системы как сервиса;
- ✓ С каждой заявки водитель оплачивает процент в диспетчерскую службу;

- ✓ Оплата заказа производится водителю (наличный или безналичный расчет);

2.4 Перечень пользователей проекта

- Диспетчер – принятие и добавление новой заявки в систему;
- Водитель – выполнение заявки;
- Клиент – создание новой заявки.

2.5 Диаграмма классов предметной области

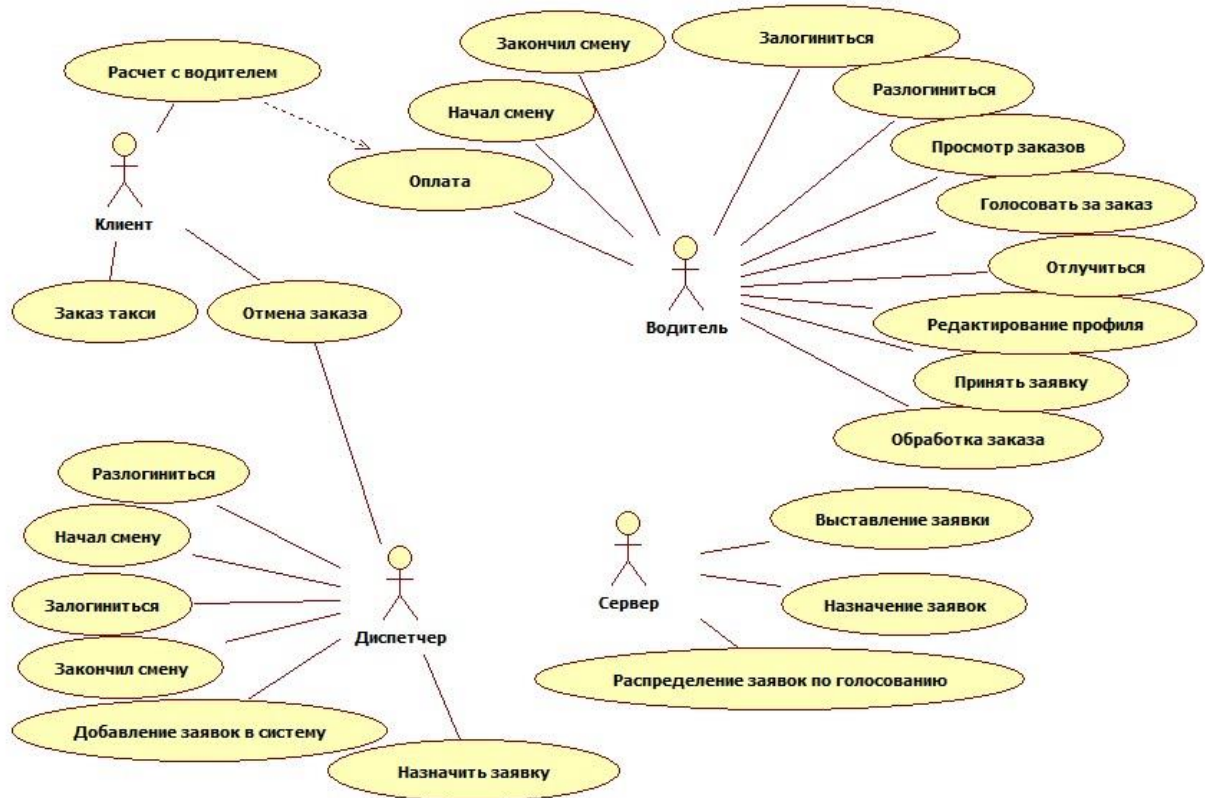


3 Модели прецедентов и спецификации требований

3.1 Актеры (роли) и прецеденты

Роли имеют все пользователи системы. В качестве псевдороль выступают внутренние процессы, управляемые внешними событиями и

другими прецедентами.



3.2 Особенности и требования к реализации

- Данные о входящей заявке должны записываться в электронный журнал.
- Для водителей, система распределения заявок и приложение должны быть максимально простыми.
- Голосование за заявку заканчивается, когда количество проголосовавших превысит 4 человека или время голосования превысит 10 минут.
- При выполнении заказа время ожидания клиента не должно превышать 10 минут от времени заказа. Дальнейшее ожидание возможно с подтверждением клиента и оплатой времени ожидания.
- Возможность оплаты заказа как наличными, так и с помощью безналичного расчета.
- Надежность.
 - При отключении питания – иметь резервный источник питания.
 - При пропадании связи с сервером требуется иметь локальную копию части БД, распределении заявок будет происходить вручную диспетчером (по радиии или телефону).
- Безопасность.

- Диспетчер. Должен иметь логин и пароль для входа в систему.
- Водитель. Ему выдается необходимое оборудование, а так же логин и пароль для входа в приложение. При увольнении логин становится недействительным, оборудование изымается.

3.3 Пример сценария

Наименование прецедента. Обработка заказа.

Актеры. Водитель. Клиент.

Предусловия. Водитель залогинился и начал смену. Заказ назначен.

Цель исполнения. Доставить клиента в место назначения и получить денежные средства.

Поток событий

1. Водитель: подтверждение на выполнение заказа.	Если есть соединение с сервером, заказ помечается как «Принят на выполнение».
	Если нет соединения – водителю необходимо сообщить о подтверждении диспетчеру (по рации или телефону), который в свою очередь пометит заказ.
2. Водитель: оповещение клиента и прибытие к назначенному клиентом месту.	Если клиент пришел в назначенное время. Переход к п.3.
	Если клиента нет по истечении допустимого времени ожидания, то производится повторное оповещение клиента. При отказе или недоступности абонента переход к п 5, иначе ожидание до выхода клиента.
3. Водитель: перевозка клиента к месту назначения.	Если клиент просит изменить маршрут, то связаться с диспетчером и сделать перерасчет.
4. Водитель: прибытие к месту назначения и оплата услуг.	Если необходим безналичный расчет, подготовить терминал для оплаты (включить, ввести необходимую сумму) и выполнить расчет.
	Если оплата производится наличными, то предоставить сдачу.
5. Водитель: завершение заказа.	Если есть соединение с сервером, заказ помечается как «Завершенный» (при необходимости поставить отметку «Отказ»).
	Если соединения нет – сообщить о завершении (отмене) заказа диспетчеру.

4 Спецификации требований

4.1 Бизнес-правила

правила.водитель.параметры.начало_смены – водитель вышел на смену.

правила.водитель.параметры.конец_смены – водитель закончил смену.

правила.водитель.параметры.состояние – не на смене, свободен, на заказе, недоступен.

правила.район – место, область к которому привязываются заказы, водители.

правила.водитель.голосование – водитель может голосовать за свободный заказ.

правила.водитель.заказ.назначен – назначен ли водитель на заказ.

правила.заказ. состояние – на голосовании, назначен, выполняется, выполнен.

правила.водитель.заказ.время_ожидания_клиента – время, через которое необходимо связаться с клиентом для подтверждения заказа.

правила.водитель.заказ.выполнение.оплата – принятие оплаты.

правила.заказ.параметры.время_поступления – время поступления заказа.

правила.заказ.параметры.время_начала – время, на которое назначен заказ или текущее время.

правила.голосование.начало_голосования – отправка заказа на голосование водителям (не менее чем за пол часа до начала заказа).

правила.голосование.конец_голосования – если кол-во проголосовавших > 4 или время до начала заказа менее 10 мин.

правила.голосование.назначение.по_голосованию – заказ назначается сервером проголосовавшему водителю TODO.

правила.голосование.назначение.принудительно – заказ назначается сервером ближайшему свободному водителю.

правила.диспетчер.параметры.начало_смены – диспетчер вышел на смену.

правила.диспетчер.параметры.конец_смены – диспетчер закончил смену.

4.2 Атрибуты качества

качество.целостность.приложение_водителя – подключение к базе заказов через приложение возможно только авторизованным пользователям.

качество.доступность.приложение_водителя – работает только при подключении к сети.

4.3 Функциональные требования

функция.сервер.БД.содержание – хранится база заказов за последний месяц.

функция.водитель.заказы – водитель видит список заказов по району, в котором находится.

функция. диспетчер.заказ.принять – принятие заказа и добавление его в базу данных.

функция. диспетчер.заказ.назначение – в случае отключения сервера, диспетчер оглашает, назначает заказ по другим каналам связи.

функция.водитель.заказ.выполнение – выполнение водителем заказа.

функция.водитель.голосование – выбор водителем заказа, выполнение голосования

функция.сервер.назначение_заказа – предлагает заказ водителям по кругу. В течение 10 секунд водитель может принять заказ, по истечении времени заказ предлагается другому водителю. Заказ предлагается только один раз.

4.4 Отчеты

отчеты.формат – необходимо представление всех отчетов в форматах html, pdf и интерактивная форма

отчеты.интерактивная_форма – вывод отчета в виде экранной формы с возможностью прокрутки и позиционирования к основной форме через клик по ячейке таблицы с установкой параметров выбранной ячейки

отчеты.виды – результаты работы водителя за рабочий день, информация о выполнении заказах за текущие сутки, информация по району.

4.5 Графический интерфейс

GUI.мобильный.отчет - должен быть реализован в интерактивной форме. При ограничении размеров экрана прокрутка должна сопровождаться выделением цветом выбранных строки и столбца

GUI.desktop.отчет.интерактивная_форма- прокрутка таблицы отчета должна производиться с сохранением в панели просмотра заголовков строк и столбцов

GUI.мобильный.информация_о_заказе – при выборе заказа, выводится информация.

5 Архитектура системы

5.1 Многослойность архитектуры ПО

- слой 1 - элементы «железного» интерфейса: пиксел, кнопка мыши, символ клавиатуры. Действия и события: заливка фрагмента окна, клик мыши с координатами экрана
- слой 2 - элементы GUI: меню, кнопки, списки, текстовые поля. Действия и события: выбор пунктов меню, нажатие кнопок, выбор элемента списка, потеря и получение фокуса, клик мышью по элементу управления
- слой 3 – бизнес-логика приложения:
 - 3.1 – представление (View), интерфейса пользователя
 - 3.2 – логика бизнес-модели, описание ее поведения (Controller)
 - 3.3 – объекты бизнес-модели и основные алгоритмы, связанные с ними (Model)
 - 3.4 – средства сборки объектов бизнес-модели из табличных объектов БД;
- слой 4 – база данных (СУБД):
 - 4.1. Табличные объекты БД
 - 4.2. Библиотека программного доступа к БД
 - 4.3. Сама СУБД.
- слой 5 – данные на физическом носителе (файлы).

Приложение водителя реализовано как приложение Android. Приложение диспетчера реализовано в качестве desktop-приложения.

1. Повторное использование кода.

2. Бизнес-объекты допускают повторное использование.
3. View-представление и Controller-представление одинаково для всех пользователей.

5.2 Представление развёртывания

На данном этапе в проработке не нуждается. Предполагает узлы-серверов БД и множество клиентов с доступом через сеть.

5.3 Процедурное представление

На данном уровне не нуждается в проработке. Синхронизация параллельного доступа к БД минимальна из-за работы пользователей с независимыми данными. Количество пользователей ограничено количеством диспетчеров и количеством водителей такси, внутренний параллелизм приложения (работа с сетью, тайм-ауты) решается на этапе проектирования.

5.4 Логическое представление

Прототип интерфейса пользователя. Выбор стиля интерфейса: «кабина самолета», табличная, мини-диалог. С учетом квалификации пользователей нужно использовать максимальное число простых форм-диалогов с развитой системой подсказок. Для документирования лучше всего – **диаграмма форм (экранов)**, с указанием переходов, вводимых данных, подсказок (строится на основе прецедентов).

5.5 Реализационное представление

Самое сложное, т.к. касается «внутренних проблем», не отраженных в прецедентах. Делается на основе анализа требований (в основном, нефункциональных). Вопрос «**как**» наиболее уязвим для формального проектирования.

Вкратце сформулируем принятое архитектурное решение:

- Можно использовать стандартный дистанционный доступ к БД через драйвер JDBC в режиме толстого клиента. Пакет доступа данных обеспечивает необходимый сервис на уровне произвольных SQL-запросов;

- Следующий слой – объекты таблиц БД – строится на основе базового класса **DB_Item**, обеспечивающего связку «производный класс – таблица БД – запись двоичного файла», что позволяет работать с произвольной физической моделью путем написания классов-наследников практически без функционала. Т.к. большинство сущностей имеют имена, следует упомянуть еще один базовый класс **DBCodeItems**;
- Слой бизнес-модели «собирает» из объектов физической модели необходимые логические представления. Это – объекты, необходимые для составления отчетов. Единственный важный класс на этом уровне – логический объект «смена», интегрирующий все данные, необходимые для текущей смены, выбранные из БД (например, **диспетчер, водители такси, заказы, оплата**). Эти данные образуют объект класса «журнал», который обеспечивает автономную работу в проблемных ситуациях и синхронизируется с основной БД. Объекты-журналы сохраняются в локальных файлах;
- Слой внешнего представления. При наличии сложной диаграммы форм (экранов) лучше всего интегрировать их через контроллер, обеспечивающий нужную последовательность состояний и событий для классов-представлений.

5.6 Параллелизм. Синхронизация

Специальных требований к параллелизму не предъявляется. В представлениях (View) исполнение продолжительных операций должно проходить в фоновом режиме, при этом элементы управления не должны быть заблокированы, но и не позволяют выполнять действия над данными, обрабатываемыми в фоновом режиме («мягкая» блокировка GIU).

5.7 Взаимодействие и представление (MVC)

Приложение водителя является мобильным(Android). При отсутствии соединения с сервером не работает.

Приложение диспетчера является desktop-приложением. При отсутствии соединения с БД приложения должны использовать источники данных локальные копии рейтингов. Этот функционал прозрачен для уровня представлений и реализован в контроллере. Последний управляет переключением и синхронизацией источников данных

5.8 Диаграмма компонентов

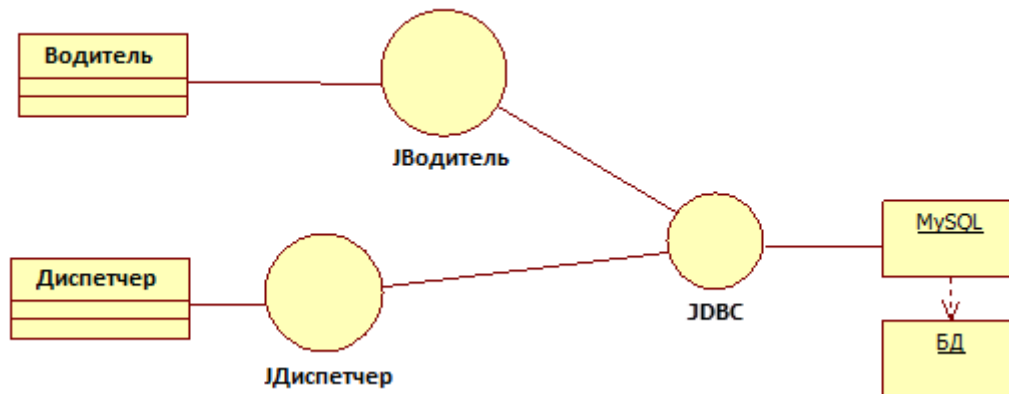
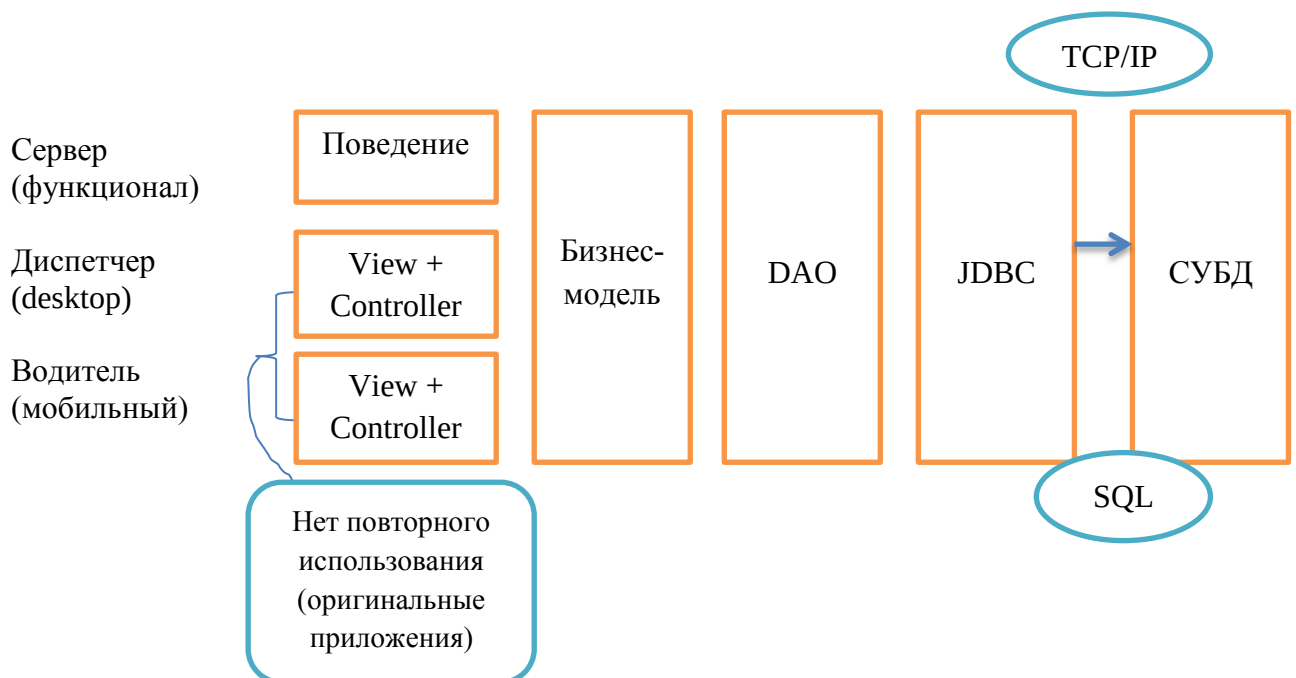


Рисунок 1. Диаграмма компонентов.



1. Приложения водителя и диспетчера оригинальны и не имеют повторного использования кода.
2. Сервер имеет свой функционал и свое уникальное поведение.
3. Приложения являются тонким клиентом. Работа с базой данных осуществляется через сервер, через запросы SQL.

6 Прототип программной реализации

6.1 База данных

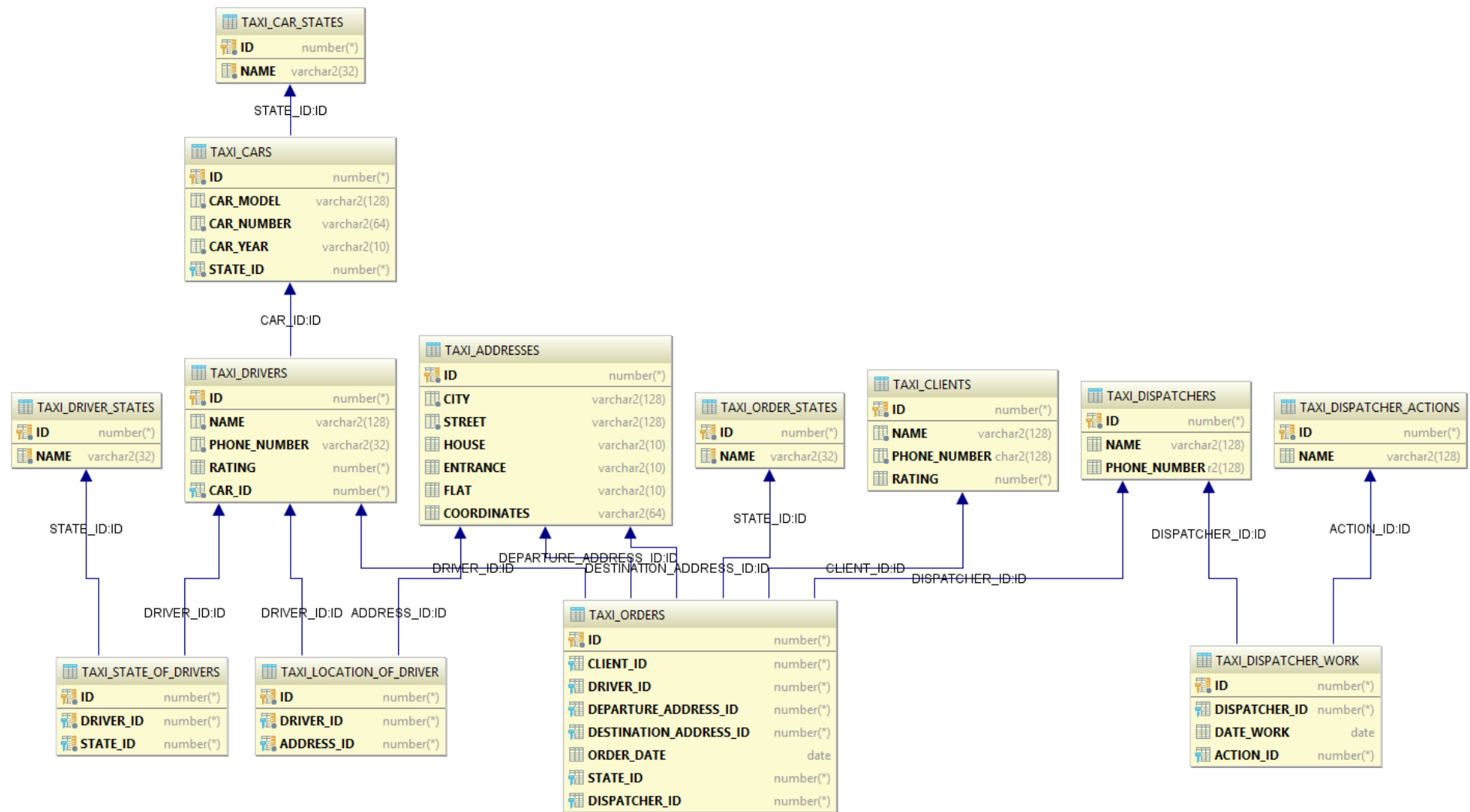
В качестве базы данных было решено использовать БД Oracle. В базе данных будет храниться вся информация о водителях и заявках. Было решено создать следующие таблицы:

- Drivers – таблица справочник для хранения всей информации о водителях. Таблица включает следующие поля:
 - ID – уникальный номер
 - NAME – ФИО водителя
 - CAR_ID – ID машины
 - PHONE_NUMBER – номер телефона водителя
 - RATING – рейтинг, оценка водителя клиентами
- Driver_States – таблица справочник, которая хранит возможные состояния водителя
 - ID – уникальный номер
 - NAME – название состояния
- Car_Status – таблица справочник, которая хранит возможные статусы машины
 - ID – уникальный номер
 - NAME – название статуса
- Order_States – таблица справочник, которая хранит возможные состояния заказа
 - ID – уникальный номер
 - NAME – название состояния
- Cars – таблица справочник, которая хранит всю информацию о машинах
 - ID – уникальный номер
 - CAR_MODEL – модель автомобиля
 - CAR_NUMBER – номер автомобиля
 - CAR_YEAR – год выпуска автомобиля
 - STATUS_ID – статус машины

- ADDRESSES– таблица справочник, которая хранит адреса и координаты
 - ID – уникальный номер
 - CITY – город
 - STREET – улица
 - HOUSE – дом
 - ENTRANCE – подъезд
 - FLAT – квартира
 - COORDINATES – координаты
- LOCATION_OF_DRIVER – таблица, в которой хранятся данные о местоположении всех водителей
 - ID – уникальный номер
 - DRIVER_ID – уникальный номер водителя
 - ADDRESS_ID – уникальный номер адреса
- CLIENTS– таблица справочник, которая хранит все данные о клиентах
 - ID – уникальный номер
 - NAME – ФИО клиента
 - PHONE_NUMBER – телефонный номер клиента
 - RATING – рейтинг клиента, который формируют водители
- DISPATCHERS – таблица справочник, которая хранит все данные о диспетчерах
 - ID – уникальный номер
 - NAME – ФИО диспетчера
 - PHONE_NUMBER – телефонный номер диспетчера
- DISPATCHER_ACTIONS– таблица справочник, которая хранит возможные действия диспетчера
 - ID – уникальный номер
 - NAME – название действия
- DISPATCHER_WORK– таблица для логирования, которая хранит когда и какие действия были выполнены диспетчером
 - ID – уникальный номер
 - DISPATCHER_ID – уникальный номер диспетчера
 - DATE_WORK – дата и время совершенного действия
 - ACTION_ID – уникальный номер действия
- STATE_OF_DRIVERS– таблица для хранения статусов водителей в текущий момент времени
 - ID – уникальный номер
 - DRIVER_ID – уникальный номер водителя

- STATE_ID – уникальный номер состояния
- ORDERS– таблица которая хранит информацию о всех заявках в системе
 - ID – уникальный номер
 - CLIENT_ID – уникальный номер клиента
 - DRIVER_ID – уникальный номер водителя
 - DEPARTURE_ADDRESS_ID - уникальный номер пункта отправления
 - DESTINATION_ADDRESS_ID - уникальный номер пункта назначения
 - ORDER_DATE – дата начала заказа
 - STATE_ID – состояние заказа
 - DISPATCHER_ID - уникальный номер диспетчера, если заявку назначил диспетчер

Структура базы данных выглядит следующим образом:



6.2 Структура приложения

В качестве паттерна программирования, которому придерживались во время разработки прототипа, взяли MVP – model view presenter.

Описание классов:

Model:

Address - класс–адрес, соответствует таблице в БД

Cars - класс–машина, соответствует таблице в БД

CarsStatus - класс–стату автомобиля, соответствует таблице в БД

Client - класс–клиент, соответствует таблице в БД

Dispatcher - класс–диспетчер, соответствует таблице в БД

Driver - класс–водитель, соответствует таблице в БД

DriverState - класс–состояние водителя, соответствует таблице в БД

IOrdersManager –интерфейс менеджера заказов, описывает все функции, необходимые для взаимодействия заказов, клиентов и водителей.

Order - класс–заказ, соответствует таблице в БД

OrderManager – класс менеджера заказов, реализующий интерфейс IOrdersManager. Здесь крутится основная логика программы.

OrderState - класс–состояние заказа, соответствует таблице в БД

PriceCalculator – класс ценовой калькулятор, позволяет вычислить итоговую стоимость поездки, в зависимости от проделанного расстояния, времени и статуса автомобиля.

Классы DriverRepository, OrderRepository, DispatcherRepository, ClientRepository, AddressRepository – это классы осуществляющие взаимодействие с базой данных, которые соответствуют определенной группе сущностей. Здесь реализованы все необходимые методы получения, редактирования и добавления данных.

Presenter:

ClientPresenter – класс представитель, через него осуществляется взаимодействия модели и представлений клиентов.

DispatcherPresenter – класс представитель, через него осуществляется взаимодействия модели и представлений диспетчеров.

DriverPresenter – класс представитель, через него осуществляется взаимодействия модели и представлений водителей.

View:

ClientView – класс, реализующий интерфейс IClientView.

IClientView – интерфейс, в котором описаны основные функции для взаимодействия представления клиента с моделью.

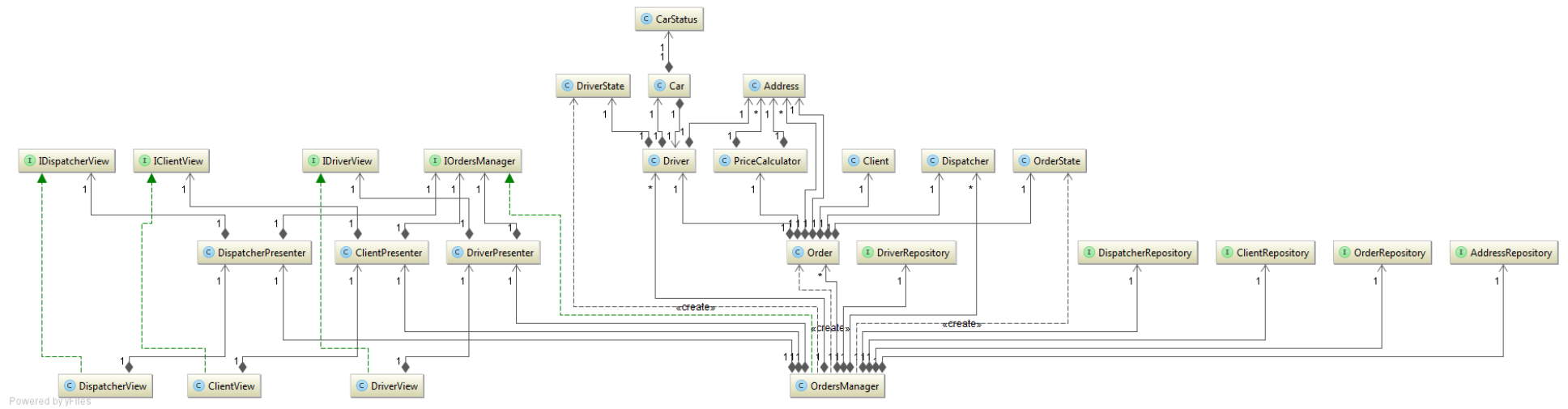
DispatcherView – класс, реализующий интерфейс IDispatcherView.

IDispatcherView – интерфейс, в котором описаны основные функции для взаимодействия представления диспетчера с моделью.

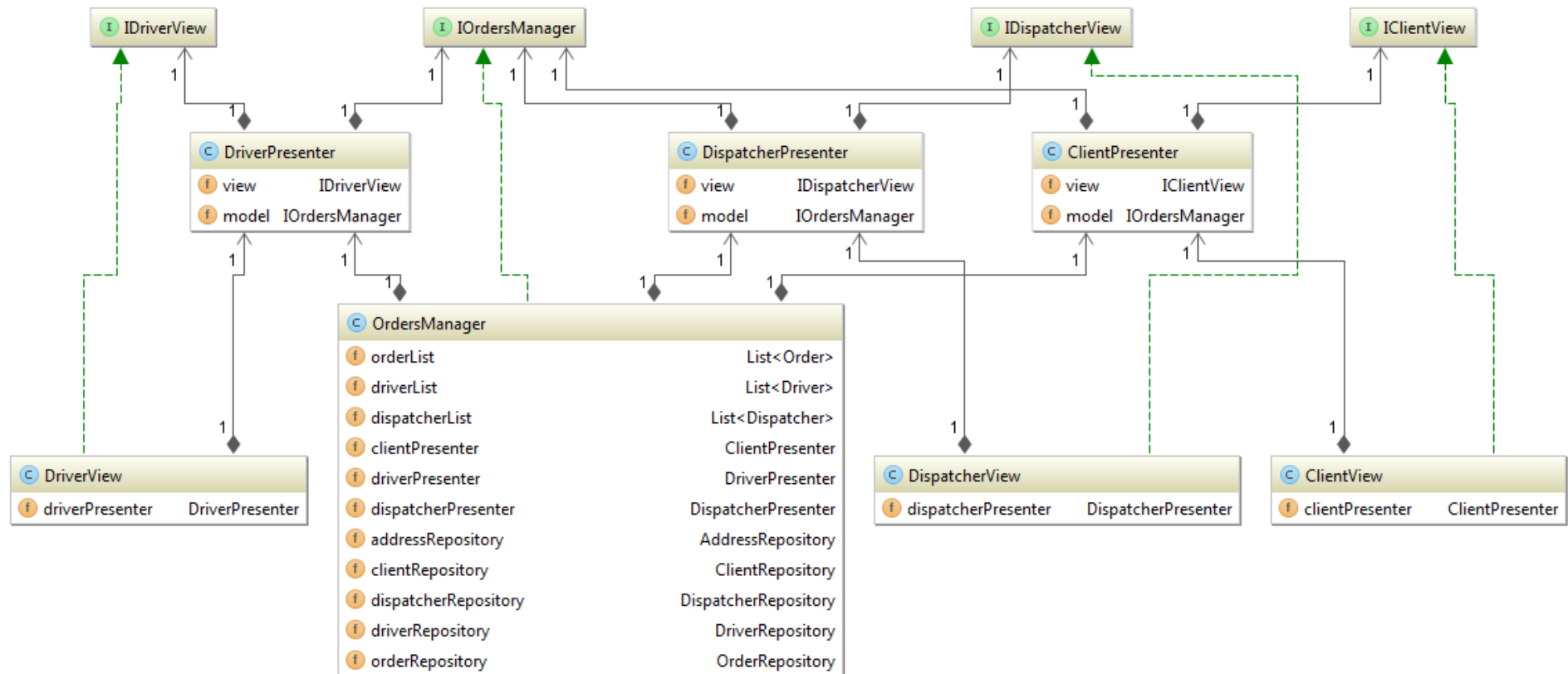
DriverView – класс, реализующий интерфейс IDriverView.

IDriverView – интерфейс, в котором описаны основные функции для взаимодействия представления водителя с моделью.

Структура всего приложения, только с названием классов:

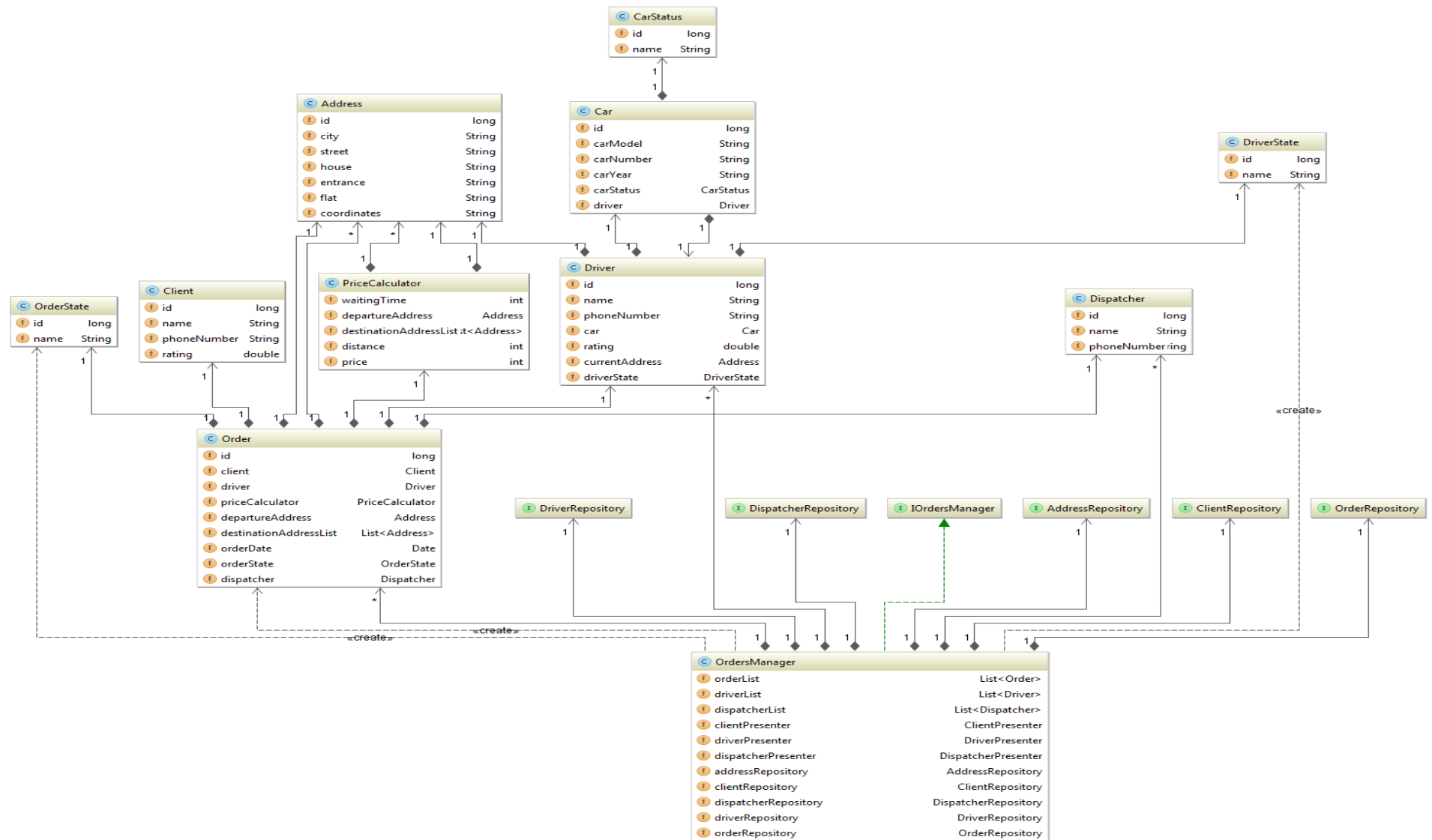


Структура классов, представляющих основную логику и взаимодействие классов паттерна MVP:



Powered by yFiles

Структура классов модели:



Заключение.

В ходе выполнения курсовой работы был реализован прототип серверной части приложения Диспетчера службы такси. Прототип реализован на основе паттерна программирования MVP. В качестве целевой БД была использована БД Oracle.

Прототип представляет собой лишь каркас реального приложения. Созданы основные классы всех сущностей, а также классы менеджеры для взаимодействия основных сущностей. Также представлены основные функции для взаимодействия сущностей и показано взаимодействие сущностей на уровне вызова объектов из соответствующих функций.

Также реализован прототип реальной базы данных. Прописаны все необходимые ограничения и зависимости. База данных представлена без наполнения данными.

Список литературы

1. JavaStudy. Spring - [Электронный ресурс]. Режим доступа: <http://javastudy.ru/frameworks/spring/>
2. JavaStudy. Spring Core- [Электронный ресурс]. Режим доступа: <http://cpp- http://javastudy.ru/frameworks/spring/spring-core/>
3. JavaStudy. Spring MVC — основные понятия, архитектура - [Электронный ресурс]. Режим доступа: <http://javastudy.ru/spring-mvc/spring-mvc-basic/>
4. JavaStudy. Spring DATA- [Электронный ресурс]. Режим доступа: <http://javastudy.ru/frameworks/spring/spring-data/>
5. Web Creator. MVC — модель-представление-контроллер - [Электронный ресурс]. Режим доступа: <https://web-creator.ru/articles/mvc>
6. ruseller.com. Концепция MVC - [Электронный ресурс]. Режим доступа: <http://ruseller.com/lessons.php?id=666>
7. spring-source.ru. Что такое JPA? - [Электронный ресурс]. Режим доступа: https://spring-source.ru/docs_simple.php?type=manual&theme=docs_simple&docs_simple=chap03_p02

Листинг программы

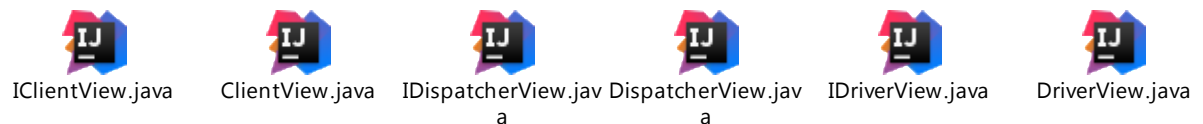
Model:



Presenter:



View:



База данных:

```
CREATE TABLE TAXI_ADDRESSES (  
  id          NUMBER PRIMARY KEY,  
  city        VARCHAR2(128) NOT NULL,  
  street      VARCHAR2(128) NOT NULL,  
  house       VARCHAR2(10),  
  entrance    VARCHAR2(10),  
  flat        VARCHAR2(10),  
  coordinates VARCHAR2(64)  
);  
  
CREATE TABLE TAXI_DRIVER_STATES (  
  id   NUMBER PRIMARY KEY,  
  name VARCHAR2(32) NOT NULL UNIQUE  
);  
  
CREATE TABLE TAXI_CAR_STATES (  
  id   NUMBER PRIMARY KEY,  
  name VARCHAR2(32) NOT NULL UNIQUE  
);  
  
CREATE TABLE TAXI_ORDER_STATES (  
  id   NUMBER PRIMARY KEY,  
  name VARCHAR2(32) NOT NULL UNIQUE  
);  
  
CREATE TABLE TAXI_CARS (  
  ID          NUMBER PRIMARY KEY,  
  CAR_MODEL   VARCHAR2(128) NOT NULL,  
  CAR_NUMBER  VARCHAR2(64)  NOT NULL,  
  CAR_YEAR    VARCHAR2(10)  NOT NULL,
```

```

        STATE_ID    NUMBER          NOT NULL,
        FOREIGN KEY (STATE_ID) REFERENCES TAXI_CAR_STATES (id)
    );

CREATE TABLE TAXI_DRIVERS (
    id              NUMBER PRIMARY KEY,
    name            VARCHAR2(128) NOT NULL,
    phone_number    VARCHAR2(32)  NOT NULL,
    rating          NUMBER,
    car_id          NUMBER          NOT NULL,
    FOREIGN KEY (car_id) REFERENCES TAXI_CARS (id)
);

CREATE TABLE TAXI_LOCATION_OF_DRIVER (
    id              NUMBER PRIMARY KEY,
    driver_id       NUMBER NOT NULL,
    address_id      NUMBER NOT NULL,
    UNIQUE (driver_id, address_id),
    FOREIGN KEY (driver_id) REFERENCES TAXI_DRIVERS (id),
    FOREIGN KEY (address_id) REFERENCES TAXI_ADDRESSES (id)
);

CREATE TABLE TAXI_STATE_OF_DRIVERS (
    id              NUMBER PRIMARY KEY,
    driver_id       NUMBER NOT NULL,
    state_id        NUMBER NOT NULL,
    UNIQUE (driver_id, state_id),
    FOREIGN KEY (driver_id) REFERENCES TAXI_DRIVERS (id),
    FOREIGN KEY (state_id) REFERENCES TAXI_DRIVER_STATES (id)
);

CREATE TABLE TAXI_CLIENTS (
    ID              NUMBER PRIMARY KEY,
    NAME            VARCHAR2(128) NOT NULL,
    PHONE_NUMBER    VARCHAR2(128) NOT NULL,
    RATING          NUMBER
);

CREATE TABLE TAXI_DISPATCHERS (
    ID              NUMBER PRIMARY KEY,
    NAME            VARCHAR2(128),
    PHONE_NUMBER    VARCHAR2(128)
);

CREATE TABLE TAXI_DISPATCHER_ACTIONS (
    ID              NUMBER PRIMARY KEY,
    NAME            VARCHAR2(128)
);

CREATE TABLE TAXI_DISPATCHER_WORK (
    ID              NUMBER PRIMARY KEY,
    DISPATCHER_ID   NUMBER,
    DATE_WORK       DATE,
    ACTION_ID       NUMBER,
    FOREIGN KEY (DISPATCHER_ID) REFERENCES TAXI_DISPATCHERS (id),
    FOREIGN KEY (ACTION_ID) REFERENCES TAXI_DISPATCHER_ACTIONS (id)
);

CREATE TABLE TAXI_ORDERS (
    id              NUMBER PRIMARY KEY,
    CLIENT_ID       NUMBER,
    DRIVER_ID       NUMBER,

```

```

DEPARTURE_ADDRESS_ID    NUMBER,
DESTINATION_ADDRESS_ID  NUMBER,
ORDER_DATE              DATE,
STATE_ID                NUMBER,
DISPATCHER_ID          NUMBER,
FOREIGN KEY (driver_id) REFERENCES TAXI_DRIVERS (id),
FOREIGN KEY (CLIENT_ID) REFERENCES TAXI_CLIENTS (id),
FOREIGN KEY (DEPARTURE_ADDRESS_ID) REFERENCES TAXI_ADDRESSES (id),
FOREIGN KEY (DESTINATION_ADDRESS_ID) REFERENCES TAXI_ADDRESSES (id),
FOREIGN KEY (STATE_ID) REFERENCES TAXI_ORDER_STATES (id),
FOREIGN KEY (DISPATCHER_ID) REFERENCES TAXI_DISPATCHERS (id)
);

COMMIT;

```